软工F模块开发

写在最前面

一、共同要求

1. 这次要完成什么

2. 必须遵守的统一规范

2.1 技术与目录规范

2.2 后端代码结构统一

2.3 接口风格统一

2.4 类型、校验、测试统一

2.5 Git 和分支统一

3. 必须统一的业务口径

3.1 模块 F 依赖哪些外部模块

3.2 数据库口径：以 Score 和 ScoreModificationLog 为核心

3.3 成绩状态统一

3.4 成绩修改统一口径

3.5 GPA、学分进展、排名都动态算

3.6 成绩修改是敏感操作，必须记日志

4. 共同的开发顺序

第 1 阶段：F1 + F2 + F3 先定后端口径

第 2 阶段：F4 + F5 接前端页面

第 3 阶段：F6 统一收口

二、每位组员直接看自己的部分

F1：你负责后端主流程——教师成绩录入

1. 你的任务目标

2. 你负责的代码范围

3. 你要依赖的数据

4. 你必须做出来的接口

4.1 获取某开课下的成绩录入列表

4.2 保存草稿

4.3 提交成绩

5. 你的权限约束

6. 你的交付物

7. 你完成后的验收标准

F2：你负责后端审批流——修改申请、审批、审计日志

1. 你的任务目标

2. 你负责的代码范围

3. 你必须做出来的接口

3.1 教师发起修改申请
3.2 获取待处理申请列表
3.3 审批通过
3.4 审批驳回
3.5 查看修改日志
4. 你必须守住的规则
5. 你要特别注意的字段设计
6. 你的交付物
7. 你完成后的验收标准
F3：你负责后端查询与统计——学生查询、GPA、成绩分析
1. 你的任务目标
2. 你负责的代码范围
3. 你必须做出来的接口
3.1 查询本人成绩列表
3.2 查询本人 GPA 与学分摘要
3.3 查询某学生成绩摘要
3.4 查询某门课成绩分析
3.5 查询某学生个人分析
4. 你必须守住的规则
5. 你需要先和谁对字段
6. 你的交付物
7. 你完成后的验收标准
F4：你负责前端教师页——成绩录入、提交、申请改分
1. 你的任务目标
2. 你负责的代码范围
3. 你要做的页面能力
4. 你必须统一的前端规则
5. 你联调时优先找谁
6. 你的交付物
7. 你完成后的验收标准
F5：你负责前端学生页——成绩查询、GPA、个人分析
1. 你的任务目标
2. 你负责的代码范围
3. 你要做的页面能力
4. 你必须统一的展示规则
5. 你联调时优先找谁
6. 你的交付物
7. 你完成后的验收标准
F6：你负责集成收口——管理员审批页、文档、测试、联调

1. 你的任务目标
2. 你负责的代码范围
3. 你要做的事情
4. 你必须产出的最终文档
5. 你要特别关注的系统级问题
6. 你的交付物
7. 你完成后的验收标准

三、统一接口标准

1. 通用对象
1.1 ScoreItem
1.2 StudentScoreSummary
1.3 CourseScoreAnalytics
2. 教师端接口
2.1 获取某开课下的成绩录入列表
2.2 保存草稿
2.3 提交成绩
3. 修改申请与审批接口
3.1 教师发起修改申请
3.2 获取待处理申请列表
3.3 审批通过
3.4 审批驳回
3.5 获取修改日志
4. 学生端接口
4.1 查询本人成绩
4.2 查询本人 GPA 与学分摘要
4.3 查询某学生成绩摘要
5. 分析接口
5.1 查询课程成绩分析
5.2 查询学生个人分析

四、统一权限矩阵

五、统一联调要求

1. 先对字段，再联调
2. 认证载荷至少要有这些字段
3. 总评 / 绩点规则不要散落在多个页面

六、各自的分支建议

七、最终验收清单

1. 功能验收
2. 技术验收
3. 文档验收

八、组长最后怎么发这份文档

写在最前面

PS: 大组仓库：https://github.com/EmergentTwilight/Smart-Teaching-Service-System

关于每人如何**开分支&提交分支**：见part. 六

开发顺序：见 part. **一、4. 共同的开发顺序**

推荐开发工作流：

1. 克隆仓库到本地，本文档的pdf移动到仓库根目录
2. 每个人分别开自己的分支
3. 拷打agent(推荐codex, claude, 半自动有cursor, copilot, etc.)完成自己的部分

分工：（大家可以把名字写到最后来领锅。领完就可以直接开始啦~ 因为是让GPT分的所以工作量应该都差不多（汗），但是DDL不太一样，见下）如果有来不及的可以和组长换一下

初定DDL是本周三4.22 F1/2/3模块定好后端口径，然后周四周五F4/5接前端，周六F6组合成一个分支检查功能。如果有问题请积极提出💡

F1 高诗奇

F2 汪昕

F3 黄哲

F4 娄思妤

F5 张安潼

F6 周小灵

一、共同要求

1. 这次要完成什么

模块 F 是 **成绩管理（Score Management）**，本次要完成的核心能力只有四类：

1. 教师录入、保存、提交成绩
2. 已提交成绩的修改申请、审批与日志追踪
3. 学生查询本人成绩、GPA、学分进展
4. 课程维度 / 学生维度的成绩分析与图表展示

这四类功能来自项目要求中的模块 F 原始需求；其中“成绩修改要进行控制”“学生能看成绩和学分进展”“课程和个人都要能做统计分析”是这次实现的主线。□filecite□turn0file0□L21-L27□ □filecite□turn0file1□L14-L16□

2. 必须遵守的统一规范

2.1 技术与目录规范

遵守项目统一技术栈和 monorepo 目录约定：

- 前端：React 19 + TypeScript 5.7 + Ant Design 5
- 后端：Node.js 22 + Express 5 + Prisma 6 + PostgreSQL 18
- 前端模块目录：`frontend/src/modules/score-management/`
- 后端模块目录：`backend/src/modules/score-management/`
- 模块接口文档目录：`docs/apis/F-score-management.md` 【filecite【turn0file0】L9-L18】 【filecite【turn0file0】L32-L50】 【filecite【turn0file0】L113-L121】

2.2 后端代码结构统一

后端模块统一按下面这种方式组织：

- `*.controller.ts`：处理 HTTP 请求
- `*.service.ts`：写业务逻辑
- `*.routes.ts`：注册路由
- `*.types.ts`：类型定义
- `*.schemas.ts`：Zod 校验（可选但强烈建议）【filecite【turn0file0】L51-L60】

2.3 接口风格统一

所有接口都必须遵守项目统一 API 规范：

- RESTful 风格
- 成功返回：`{ code, message, data }`
- 分页返回：`{ code, message, data: { items, pagination }, request_id? }`
- 错误返回：`{ code, message, errors?, request_id? }`
- 常用状态码：200 / 201 / 204 / 400 / 401 / 403 / 404 / 409 / 422 / 429 / 500 【filecite【turn0file0】L75-L105】

2.4 类型、校验、测试统一

要统一遵守这些底线：

- 不要用 `any`
- 输入参数统一用 Zod 做校验
- 注释用中文，解释“为什么”，不要只解释“做什么”
- 新功能必须补测试
- 单元测试覆盖率目标 ≥ 80% 【filecite【turn0file0】L106-L143】

2.5 Git 和分支统一

都从 `dev/F` 出功能分支，命名风格统一按项目规范来：

- `feat/F-xxx-0420`
- `fix/F-xxx-0420`
- `docs/F-xxx-0420`

Commit 格式统一：

Plain Text

模块 F 的 scope 固定是 `F`。□filecite□turn0file0□L61-L74□

3. 必须统一的业务口径

这个部分是整个模块最容易乱掉的地方。任何组员都不能各写各的。

3.1 模块 F 依赖哪些外部模块

这次不是孤立写模块 F，而是要和其他模块衔接：

- 依赖 A 组：认证、用户角色、权限控制、SystemLog
- 依赖 C 组：Course / CourseOffering / Enrollment / Semester / Curriculum / CurriculumCourse
- 和 E 组暂时不强耦合，但接口要预留后续“从测试结果导入部分成绩”的扩展位 □filecite□turn0file1□L2-L9□ □filecite□turn0file1□L14-L16□

3.2 数据库口径：以 Score 和 ScoreModificationLog 为核心

数据库设计文档里，模块 F 的核心表是：

- `Score`
- `ScoreModificationLog`

并且文档明确说：**GPA 和排名不单独建表存储，每次根据 Score 动态计算。** 所以这次实现必须按这个口径来，不要再新做一套持久化 GPA 逻辑。□filecite□turn0file1□L14-L16□

3.3 成绩状态统一

统一使用这三个状态：

- `DRAFT`：草稿
- `SUBMITTED`：教师已提交
- `CONFIRMED`：管理员确认完成（如果你们决定保留二次确认）

统一规则：

- 草稿阶段可以正常编辑

- 一旦提交，教师不能再直接改分
- 提交后如果要改分，必须走“修改申请 → 管理员审批”

3.4 成绩修改统一口径

数据库文档里 `Score` 已经有这些字段：

- `modification_request`
- `modified_at`
- `modified_by`

并且有单独的 `ScoreModificationLog` 表，所以本次建议直接按这个结构落地：

- 教师发起改分时，不直接更新成绩
- 把申请内容写到 `modificationRequest`
- 管理员审批通过后，正式写回 `Score`
- 同时写 `ScoreModificationLog`
- 同时写 `SystemLog`
- 最后清空 `modificationRequest` 【filecite【turn0file1】L14-L16】

3.5 GPA、学分进展、排名都动态算

统一要求：

- GPA 不依赖历史表，不做静态缓存口径
- 学分进展基于 `Curriculum/CurriculumCourse + Score + Course + Enrollment`
- 排名先支持课程内排名
- 前后端如果都需要算，**以后端为准，前端只负责展示**

3.6 成绩修改是敏感操作，必须记日志

开发规范里明确列出了敏感操作日志，其中包含 **成绩修改**。所以只要发生审批通过后的正式改分，就必须写 `SystemLog`。【filecite【turn0file0】L156-L164】

4. 共同的开发顺序

不要 6 个人一上来都去堆页面。模块 F 先把后端口径定死，再做前端。

推荐顺序：

第 1 阶段：F1 + F2 + F3 先定后端口径

先完成：

1. 成绩录入列表 / 保存草稿 / 提交成绩

2. 修改申请 / 审批 / 日志
3. 学生查询 / GPA summary / analytics

第 2 阶段：F4 + F5 接前端页面

先用已约定 DTO 和接口名把页面壳子搭出来，再联调真实接口。

第 3 阶段：F6 统一收口

负责：

1. 管理员审批页
2. 路由接入
3. 文档输出
4. E2E 回归
5. 最终演示链路打通

二、每位组员直接看自己的部分

F1：你负责后端主流程——教师成绩录入

1. 你的任务目标

你负责做出教师侧最核心的闭环：

- 进入某门课的成绩录入页
- 拉出这门课下所有学生
- 展示已有成绩或空白占位记录
- 支持批量保存草稿
- 支持批量提交成绩

你的模块是整个 F 的入口，没有你，前端教师页没法联调。

2. 你负责的代码范围

建议你新建并维护这些文件：

Plain Text

3. 你要依赖的数据

你要主要依赖：

- `CourseOffering`
- `Enrollment`
- `Student`
- `Course`
- `Semester`
- `Score`

统一原则：

- 模块 F 不自己维护选课关系
- 一律以 `Enrollment` 为准生成成绩录入名单

4. 你必须做出来的接口

4.1 获取某开课下的成绩录入列表

```
HTTP
```

要求：

- 支持分页
- 支持按学号 / 姓名筛选
- 支持按状态筛选
- 如果某个学生还没有 `Score` 记录，后端也要返回“可录入占位项”，不要把拼装工作丢给前端

4.2 保存草稿

```
HTTP
```

要求：

- 支持批量 upsert
- 只允许保存为 `DRAFT`
- 分数统一校验 `0 ~ 100`
- `totalScore` 允许前端传，但如果和后端规则冲突，以后端重算结果为准

4.3 提交成绩

```
HTTP
```

要求：

- 只允许提交当前课程下的成绩
- 提交后状态改为 `SUBMITTED`

- 提交后普通编辑接口必须失效

5. 你的权限约束

你必须保证：

- 教师只能操作自己任课的 `CourseOffering`
- 管理员可以查看或代操作，但教师不能越权访问别人的课程

6. 你的交付物

你最终要交付：

- route / controller / service / types / schemas
- 列表 DTO
- 草稿保存逻辑
- 提交逻辑
- 单元测试
- 给 F4 一份字段说明

7. 你完成后的验收标准

- 教师能看到自己课程下的学生成绩列表
- 草稿可以反复保存覆盖
- 提交后不能再通过录入接口直接改分
- 接口响应格式完全符合项目统一规范

F2：你负责后端审批流——修改申请、审批、审计日志

1. 你的任务目标

你负责做出“提交后改分”的完整闭环。这部分是模块 F 最敏感、最容易出 bug 的部分。

2. 你负责的代码范围

```
Plain Text
```

3. 你必须做出来的接口

3.1 教师发起修改申请

```
HTTP
```

要求：

- 仅允许 `SUBMITTED/CONFIRMED` 成绩发起申请
- 不直接更新 `Score`
- 申请内容写入 `modificationRequest`

3.2 获取待处理申请列表

```
HTTP
```

要求：

- 仅管理员可用
- 支持分页
- 支持按课程、教师筛选

3.3 审批通过

```
HTTP
```

要求：

- 读取申请内容
- 更新 `Score`
- 写 `ScoreModificationLog`
- 写 `SystemLog`
- 更新 `modifiedAt / modifiedBy`
- 清空 `modificationRequest`

3.4 审批驳回

```
HTTP
```

要求：

- 原成绩不能变化
- 清空 `modificationRequest`
- 写 `SystemLog`

3.5 查看修改日志

```
HTTP
```

4. 你必须守住的规则

你要严格区分两种修改：

- `DRAFT` 阶段的修改：不需要申请，由 F1 负责的录入流程直接处理
- `SUBMITTED/CONFIRMED` 阶段的修改：必须走你这套审批流

5. 你要特别注意的字段设计

建议你把 `modificationRequest` 统一成下面这种 JSON 结构，避免后面前后端各写各的：

JSON

6. 你的交付物

- 申请与审批接口
- 审计日志逻辑
- 权限校验
- 单元测试
- 给 F6 一份审批页字段说明

7. 你完成后的验收标准

- 教师无法直接修改已提交成绩
- 管理员审批后能追溯前后差异
- 驳回时原始成绩保持不变
- SystemLog 和 ScoreModificationLog 都能查到记录

F3：你负责后端查询与统计——学生查询、GPA、成绩分析

1. 你的任务目标

你负责所有“看成绩”和“算统计”的后端逻辑。你的东西决定学生页和分析页能不能做出来。

2. 你负责的代码范围

Plain Text

3. 你必须做出来的接口

3.1 查询本人成绩列表

HTTP

要求：

- 支持按学期筛选
- 支持按课程关键词筛选
- 支持分页

3.2 查询本人 GPA 与学分摘要

HTTP

要求至少返回：

- GPA
- 平均分
- 已修学分
- 通过学分
- 应修学分
- 已完成课程数 / 未通过课程数

3.3 查询某学生成绩摘要

HTTP

3.4 查询某门课成绩分析

HTTP

要求至少返回：

- 平均分
- 最高分 / 最低分
- 及格人数 / 不及格人数
- 成绩分布
- 课程内排名 Top10

3.5 查询某学生个人分析

HTTP

要求至少返回：

- 按学期的 GPA / 均分趋势
- 分数段分布

- 不同课程类型的学分完成情况

4. 你必须守住的规则

- GPA 和排名动态计算，不依赖独立历史表
- 学生只能看自己
- 教师只能看自己任课课程的课程分析
- 管理员可以看学生和课程两种视角

5. 你需要先和谁对字段

你开始前先找：

- F1：确认 `ScoreItem` 字段口径
- C 组：确认 `CourseOffering / Enrollment / CurriculumCourse` 的字段名已经稳定
- F5：提前对齐 summary 和 analytics 的接口字段

6. 你的交付物

- 查询接口
- 统计接口
- GPA / 学分 / 排名计算逻辑
- 单元测试
- 给 F5 一份接口返回示例

7. 你完成后的验收标准

- 学生只能看到自己的成绩与统计
- 教师能看到课程维度分析
- 所有统计口径以后端为准且前后一致

F4：你负责前端教师页——成绩录入、提交、申请改分

1. 你的任务目标

你负责把 F1 和 F2 的教师侧接口做成可操作页面。你交付的是“老师真的能用”的录入体验。

2. 你负责的代码范围

```
Plain Text
```

3. 你要做的页面能力

你要把下面这些交互做出来：

- 选择课程开设
- 拉取学生名单和成绩数据
- 表格内编辑平时 / 期中 / 期末分数
- 实时展示总评
- 批量保存草稿
- 批量提交成绩
- 对已提交成绩发起修改申请

4. 你必须统一的前端规则

- 非法分数 `<0` 或 `>100`` 直接拦截
- 录入态与只读态要明显区分
- 已提交成绩不能继续当草稿编辑
- 前端可以展示即时计算结果，但最终总评以后端返回为准

5. 你联调时优先找谁

- 先找 F1：列表、草稿、提交
- 再找 F2：修改申请弹窗与申请状态

6. 你的交付物

- 教师录入页
- API client
- React Query hooks
- 表格组件
- 改分申请弹窗

7. 你完成后的验收标准

- 页面可录入、不丢数据
- 提交后状态切换正确
- 已提交成绩只能走申请流程
- 接口异常、空数据、加载态都处理完整

F5：你负责前端学生页——成绩查询、GPA、个人分析

1. 你的任务目标

你负责学生侧体验，让学生能清楚看到：我这学期成绩怎么样、GPA 是多少、学分修到哪了。

2. 你负责的代码范围

Plain Text

3. 你要做的页面能力

你要完成：

- 查询本人成绩列表
- 按学期 / 课程名筛选
- 查看单门课成绩构成
- 展示 GPA、均分、已修学分、通过学分、培养方案进度
- 展示个人分析图表

4. 你必须统一的展示规则

- 数据口径以后端为准
- 不自己另算 GPA 当真值
- 图表只做展示，不藏业务逻辑
- 学生视图严格只展示本人数据

5. 你联调时优先找谁

- 主要找 F3
- 如果涉及成绩构成详情，也同步问 F1 的 `ScoreItem` 字段

6. 你的交付物

- 学生成绩页
- GPA 卡片
- 图表组件
- API hooks
- 异常态 / 空态 / 加载态

7. 你完成后的验收标准

- 页面上能完整查看本人成绩与统计
- 筛选与切学期可正常工作

- 图表展示和后端返回字段完全对齐

F6：你负责集成收口——管理员审批页、文档、测试、联调

1. 你的任务目标

你是模块 F 的收口位。你的工作不是“做一点边角料”，而是把大家做的东西真正拼成一个能演示、能交付的模块。

2. 你负责的代码范围

Plain Text

3. 你要做的事情

你要负责：

- 管理员审批页面
- 前后端路由接入
- 菜单接入
- E2E 测试
- 种子数据补齐
- 输出最终接口文档
- 整理联调问题清单

4. 你必须产出的最终文档

你要把最终接口标准落到：

Plain Text

文档必须带完整元数据，并在合并前更新 `last_updated_at` 和 `last_updated_by`。
filecite turn0file0 L165-L178

5. 你要特别关注的系统级问题

- 后端 routes 是否注册到主路由
- 前端页面是否挂到菜单与路由
- 测试账号角色是否能覆盖教师 / 学生 / 管理员
- 种子数据能否支撑完整演示链路
- 文档字段是否和真实接口一致

6. 你的交付物

- 管理员审批页
- 路由接入
- E2E 测试
- 最终接口文档
- 联调 checklist

7. 你完成后的验收标准

- 从登录开始，教师录入 → 提交 → 申请改分 → 管理员审批 → 学生查询，整条链路能跑通
- 文档与真实接口一致
- `dev/F` 分支可以直接做演示

三、统一接口标准

这一部分不是只给某一个人看的，是 F1/F2/F3/F4/F5/F6 联调时共同遵守的口径。

1. 通用对象

1.1 ScoreItem

Plain Text

1.2 StudentScoreSummary

Plain Text

1.3 CourseScoreAnalytics

Plain Text

2. 教师端接口

2.1 获取某开课下的成绩录入列表

```
HTTP
```

Query：

- `page?: number`
- `pageSize?: number`
- `keyword?: string`
- `status?: DRAFT | SUBMITTED | CONFIRMED`

权限：

- `teacher`：仅自己任课
- `admin / super_admin`：可查看全部

2.2 保存草稿

```
HTTP
```

Body：

```
JSON
```

2.3 提交成绩

```
HTTP
```

Body：

```
JSON
```

3. 修改申请与审批接口

3.1 教师发起修改申请

```
HTTP
```

Body：

```
JSON
```

3.2 获取待处理申请列表

HTTP

3.3 审批通过

HTTP

3.4 审批驳回

HTTP

3.5 获取修改日志

HTTP

4. 学生端接口

4.1 查询本人成绩

HTTP

4.2 查询本人 GPA 与学分摘要

HTTP

4.3 查询某学生成绩摘要

HTTP

5. 分析接口

5.1 查询课程成绩分析

HTTP

5.2 查询学生个人分析

HTTP

返回建议：

Plain Text

四、统一权限矩阵

操作	student	teacher	admin	super_admin
查询本人成绩	✅	❌	✅	✅
查询自己任课课程成绩	❌	✅	✅	✅
保存草稿	❌	✅	✅	✅
提交成绩	❌	✅	✅	✅
已提交成绩直接编辑	❌	❌	❌	❌
发起修改申请	❌	✅	✅	✅
审批修改申请	❌	❌	✅	✅
查看修改日志	仅本人相关	仅任课相关	✅	✅
查看课程成绩分析	❌	✅	✅	✅
查看学生成绩统计	仅本人	❌	✅	✅

五、统一联调要求

1. 先对字段，再联调

模块 F 联调前必须先确认这几个字段名稳定：

- `Enrollment.id`
- `Enrollment.studentId`

- `Enrollment.courseOfferingId`
- `CourseOffering.id`
- `CourseOffering.courseId`
- `CourseOffering.teacherId`
- `CourseOffering.semesterId`
- `Course.id / code / name / credits / courseType`
- `Curriculum / CurriculumCourse`

2. 认证载荷至少要有这些字段

你们需要确认 A 组的 `req.user` 至少包含：

Plain Text

否则模块 F 无法做：

- 教师任课校验
- 学生仅可看自己校验
- 管理员审批权限校验

3. 总评 / 绩点规则不要散落在多个页面

建议统一放在共享位置，例如：

Plain Text

至少统一：

- 总评计算方式
- 是否四舍五入
- 等级划分
- 绩点换算
- 通过线

如果来不及抽到 shared，就统一以后端 service 为准，前端只展示。

六、各自的分支建议

- F1：`feat/F-score-entry-backend-0420`
- F2：`feat/F-score-modification-backend-0420`
- F3：`feat/F-score-query-analytics-backend-0420`
- F4：`feat/F-score-entry-frontend-0420`

- F5：`feat/F-student-score-frontend-0420`
- F6：`feat/F-score-integration-docs-0420`

Commit 示例：

七、最终验收清单

1. 功能验收

- 教师能看到自己任课课程的成绩录入页
- 草稿可保存
- 成绩可提交
- 提交后不能直接改分
- 教师可发起修改申请
- 管理员可审批 / 驳回
- 学生可查询本人成绩
- 学生可查看 GPA 与学分进展
- 教师 / 管理员可查看课程成绩分析图表

2. 技术验收

- 所有接口使用统一响应格式
- 所有输入使用 Zod 校验
- 权限校验完整
- 成绩修改写 `SystemLog`
- 审批通过写 `ScoreModificationLog`
- 单元测试覆盖关键 service
- E2E 至少覆盖 1 条完整主链路

3. 文档验收

- `docs/apis/F-score-management.md` 已补全
- 字段名与真实接口一致
- 页面、路由、权限说明齐全
- 更新 `last_updated_at` 和 `last_updated_by`

八、组长最后怎么发这份文档

你把这份文档发出去时，可以直接跟组员说：

1. 先看“共同要求”
2. 再只看自己负责的小节
3. 今天内先把接口名、DTO、字段口径对齐
4. 写代码过程中如果发现口径冲突，优先改文档，再改代码

这样大家不会一上来就各写各的，后面联调会顺很多。